Documentation: DigplexQ

Heitor Baldo*

Contents

*hbaldo@usp.br

1 Introduction

DigplexQ is a Python package to perform computations with digraph-based complexes (directed flag complexes and path complexes). It is an "adjacency matrix-centered" package since it was designed so that the user can perform all computations just by entering an adjacency matrix as input.

This package implements several quantitative methods for the analysis of digraph-based complexes, mainly based on concepts from directed Q-Analysis [1].

2 Installation

The DigplexQ package can be installed through the PIP package manager via the `pip` command:

```
pip install digplexq
```

It is available in the PyPi repository at https://pypi.org/project/digplexq (current version 0.0.7).

3 Dependencies

The following Python packages are required:

- numpy
- scipy
- networkx
- gtda
- persim
- hodgelaplacians

4 Basic Usage

Below is a simple example of using DigplexQ:

```
from digplexq.directed_q_analysis import *
from digplexq.digraph_based_complexes import *
from digplexq.structure_based_simplicial_measures import *
from digplexq.random_digraphs import *
from digplexq.utils import *

M = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M = remove_double_edges(M) #remove double edges.

#Directed flag complex:
DFC = DirectedFlagComplex(M, "by_dimension_with_nodes")

#Maximal directed simplices:
maxsimp = MaximalSimplices(DFC)

#q-Adjacency matrix:
fast_q_adjacency_matrix(M, q=1)

#in-q-degree centrality
in_q_degree_centrality(M, q=1, results="nodes")
```

5 Modules Overview

5.1 digplexq.digraph_based_complexes

- `DirectedFlagComplex(M: NumPy matrix)`: Returns the directed flag complex of a given digraph (M must be the adjacency matrix of the digraph).
- `PathComplex(M: NumPy matrix)`: Returns the path complex of a given digraph (M must be the adjacency matrix of the digraph).

5.2 digplexq.directed_q_analysis

- `q_adjacency_matrix(DFC: array, q: int)`: Returns the q-adjacency matrix of a directed flag complex.

5.3 digplexq.discrete_curvatures

- `q_forman_ricci_curvature(M: NumPy matrix, DFC: array, edge_curv: array, node_weight: string, edge_weight: string)`: Returns the Forman-Ricci curvature of a weighted directed edge (edge_curv).
- `in_q_forman_ricci_curvature(M: NumPy matrix, DFC: array, v: int, node_weight: string, edge_weight: string)`: Returns the in-q-Forman-Ricci curvature of a vertex v.
- `out_q_forman_ricci_curvature(M: NumPy matrix, DFC: array, v: int, node_weight: string, edge_weight: string)`: Returns the out-q-Forman-Ricci curvature of a vertex v.

5.4 digplexq.persistent_homology

- `persistence_diagram(M: NumPy matrix)`: Returns the persistence diagram of a digraph.
- `wasserstein_distance(M1: NumPy matrix, M2: NumPy matrix)`: Returns the Wasserstein distance between two digraphs (M1 and M2 must be the adjacency matrices of the digraphs).
- `bottleneck_distance(M1: NumPy matrix, M2: NumPy matrix)`: Returns the bottleneck distance between two digraphs (M1 and M2 must be the adjacency matrices of the digraphs).
- `betti_distance(M1: NumPy matrix, M2: NumPy matrix)`: Returns the L2 distance between two Betti curves.
- `landscape_distance(M1: NumPy matrix, M2: NumPy matrix)`: Returns the L2 distance between two persistence landscapes.
- `silhouette_distance(M1: NumPy matrix, M2: NumPy matrix)`: Returns the L2 distance between two silhouettes.
- `persistence_image_distance(M1: NumPy matrix, M2: NumPy matrix)`: Returns the L2 distance between two Gaussian-smoothed diagrams.

5.5 digplexq.structure_based_simplicial_measures

- `average_shortest_q_walk_length(M: NumPy matrix, q: int)`: Returns the average shortest q-walk length of a digraph.
- `q_eccentricity(M: NumPy matrix, q: int)`: Returns the q-eccentricity of a digraph.
- `in_q_degree(M: NumPy matrix, q: int, i: int)`: Returns the in-q-degree of a digraph's node i.
- `out_q_degree(M: NumPy matrix, q: int, i: int)`: Returns the out-q-degree of a digraph's node i.
- `in_q_degree_centrality(M: NumPy matrix, q: int)`: Returns the in-q-degree centrality of a digraph.
- `out_q_degree_centrality(M: NumPy matrix, q: int)`: Returns the out-q-degree centrality of a digraph.
- `q_closeness_centrality(M: NumPy matrix, q: int, results: string, wf_improved: Boolean)`: Returns the q-closeness centrality of the nodes of a digraph.
- `q_harmonic_centrality(M: NumPy matrix, q: int, results: string)`: Returns the q-harmonic centrality of a digraph.
- `q_betweenness_centrality(M: NumPy matrix, q: int, results: string)`: Returns the q-betweenness centrality of the nodes of a digraph.
- `q_katz_centrality(M: NumPy matrix, q: int, results: string, alpha: float, beta: float, normalized: Boolean)`: Returns the q-Katz centrality of the nodes of a digraph.
- `global_q_reaching_centrality(M: NumPy matrix, q: int, normalized: Boolean)`: Returns the q-reaching centrality of a digraph.
- `q_efficiency(M: NumPy matrix, i: int, q: int)`: Returns the q-efficiency of a digraph's node i.
- `global_q_efficiency(M: NumPy matrix, q: int)`: Returns the global q-efficiency of a digraph.
- `average_q_clustering_coefficient(M: NumPy matrix, q: int)`: Returns the average q-clustering coefficient of a q-digraph.
- `q_communicability(M: NumPy matrix, i: int, j: int, q: int)`: Returns the q-communicability of the nodes i and j.
- `q_communicability_max(M: NumPy matrix, q: int)`: Returns the maximum q-communicability among all the nodes i and j.
- `q_returnability(M: NumPy matrix, q: int, normalized: Boolean)`: Returns the q-returnability of a digraph.

5.6 digplexq.spectrum_based_simplicial_measures

- `hodge_q_laplacian(DFC: array, q: int)`: Returns the Hodge q-Laplacian.
- `q_energy(M: NumPy matrix, q: int)`: Returns the q-energy of a digraph.
- `q_spectral_density(L: NumPy matrix, b: int)`: Returns the q-spectral density of a digraph (L must be the Hodge q-Laplacian).
- `q_spectral_entropy(L: NumPy matrix)`: Returns the q-spectral entropy of a digraph (L must be the Hodge q-Laplacian).
- `q_vonNeumann_entropy(L: NumPy matrix, b: int)`: Returns the q-von-Neumann entropy of a digraph (L must be the Hodge q-Laplacian).
- `q_KL_divergence(L1: NumPy matrix, L2: NumPy matrix, b: int)`: Returns the q-von-Neumann entropy of a digraph (L1 and L2 must be the Hodge q-Laplacians of the digraphs).
- `q_spectral_distance(L1: NumPy matrix, L2: NumPy matrix)`: Returns the q-spectral distance between two Hodge q-Laplacians.
- `q_eigenvector_centrality(M: NumPy matrix, q: int)`: Returns the q-eigenvector centrality of a digraph.

5.7 digplexq.structure_based_simplicial_distances

- `first_topological_distance(M1: NumPy matrix, M2: NumPy matrix, comp: string)`: Returns the 1st topological distance.
- `second_topological_distance(M1: NumPy matrix, M2: NumPy matrix, comp: string)`: Returns the 2nd topological distance.
- `fourth_topological_distance(M1: NumPy matrix, M2: NumPy matrix, comp: string)`: Returns the 4th topological distance.
- `fifth_topological_distance(M1: NumPy matrix, M2: NumPy matrix, comp: string)`: Returns the 5th topological distance.
- `histogram_cosine_kernel(M1: NumPy matrix, M2: NumPy matrix)`: Returns the histogram cosine kernel (HCK) between two directed flag complexes.
- `jaccard_kernel(M1: NumPy matrix, M2: NumPy matrix)`: Returns the Jaccard kernel between two directed flag complexes.

6 Examples

6.1 Structure-Based Simplicial Measures

```
M = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M = remove_double_edges(M) #remove double edges.

#in-q-degree centrality (for q=1)
in_q_degree_centrality(M, q=1, results="nodes")

#out-q-degree centrality (for q=1)
out_q_degree_centrality(M, q=1, results="nodes")
```

```
#average shortest q-walk length (for q=0)
average_shortest_q_walk_length(M, q=0)
```

6.2 Spectrum-Based Simplicial Measures

```
M1 = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M1 = remove_double_edges(M) #remove double edges.

M2 = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M2 = remove_double_edges(M) #remove double edges.

#Directed flag complexes:
DFC1 = DirectedFlagComplex(M1, "by_dimension_with_nodes")
DFC2 = DirectedFlagComplex(M2, "by_dimension_with_nodes")

#Hodge q-Laplacians (q=1):
L1 = hodge_q_laplacian(DFC1, q = 1)
L2 = hodge_q_laplacian(DFC2, q = 1)

#q-spectral entropy:
q_spectral_entropy(L1)

#q-spectral distance:
q_spectral_distance(L1, L2)
```

6.3 Structure-Based Simplicial Distances

```
M1 = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M1 = remove_double_edges(M) #remove double edges.

M2 = directed_erdos_renyi_GnM_model(20, 40, weight=False)
M2 = remove_double_edges(M) #remove double edges.

#histogram cosine kernel (HCK):
histogram$\_$cosine_kernel(M1, M2)

#Jaccard Kernel:
jaccard_kernel(M1, M2)
```

References

[1] Baldo, H. (2024). Towards a Quantitative Theory of Digraph-Based Complexes and its Applications in Brain Network Analysis. [PhD Thesis, University of São Paulo] *arXiv:2409.09862*

7 License